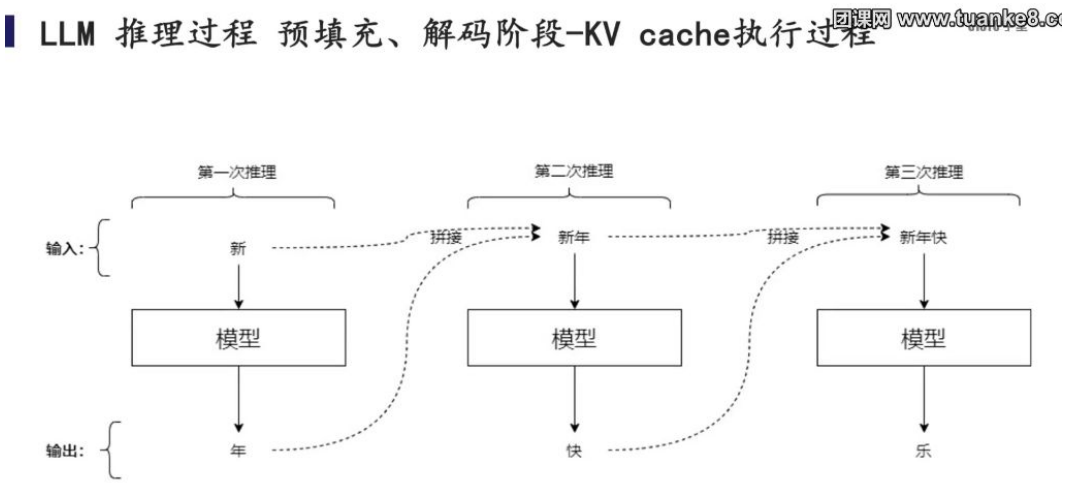
以下为AI生成的图文笔记的内容

一、知识梳理 00:00

- **注意力机制计算**：在预填充和解码阶段都会使用Q、K、V三个向量进行计算，计算过程逐个token进行。计算公式为：$softmax(\frac{QK^T}{\sqrt{dimension}})V$
- **自回归特性**：每个token的生成都依赖于前面所有token的向量计算，形成链式依赖关系

二、推理过程 00:32

1. 第一次推理 01:04



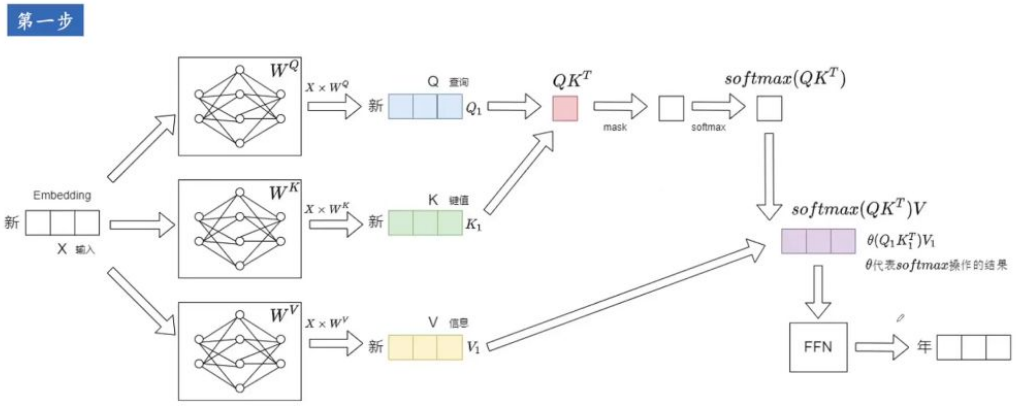
-
- **输入处理**：输入单个"新"字，模型进行词嵌入(Embedding)操作，将文字转换为高维向量
- **向量生成**：通过矩阵运算生成三个向量：
  - Q：查询向量
  - K：键向量
  - V：值向量
- **注意力计算**：执行QK^T矩阵乘法→softmax归一化→与V相乘
- **输出生成**：通过前馈神经网络(FFN)输出"年"字

2. 第二次推理 02:14

微信+Tuanbo8

LLM 推理过程 预填充、解码阶段-KV cache执行过程

51CTO学堂



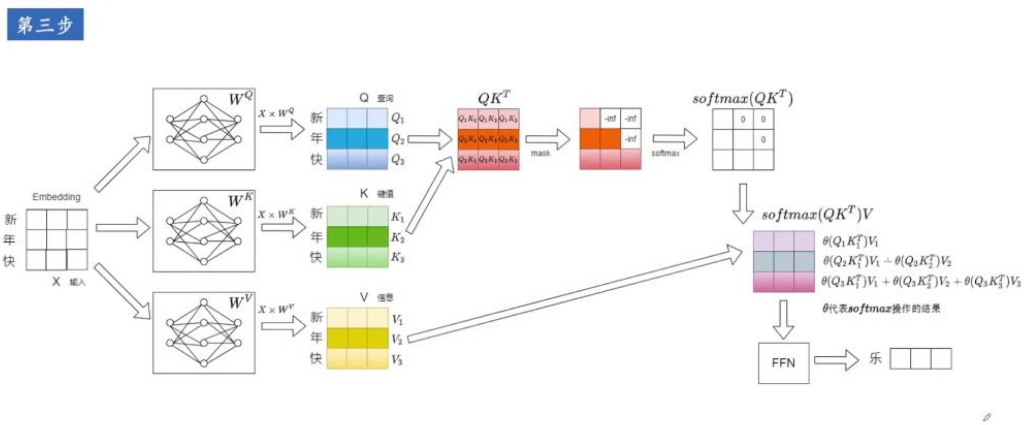
-
- **输入拼接**：将第一次生成的"新"和"年"拼接作为新输入
- **向量生成**：
  - "新"保留原有的K_1、V_1向量
  - "年"生成新的Q_2、K_2、V_2向量
- **复合计算**：注意力计算变为$\theta(Q_2K_1^T)V_1 + \theta(Q_2K_2^T)V_2$，其中$\theta$表示softmax操作
- **输出生成**：通过FFN输出"快"字

3. 第三步 07:26

微信+Tuanbo8

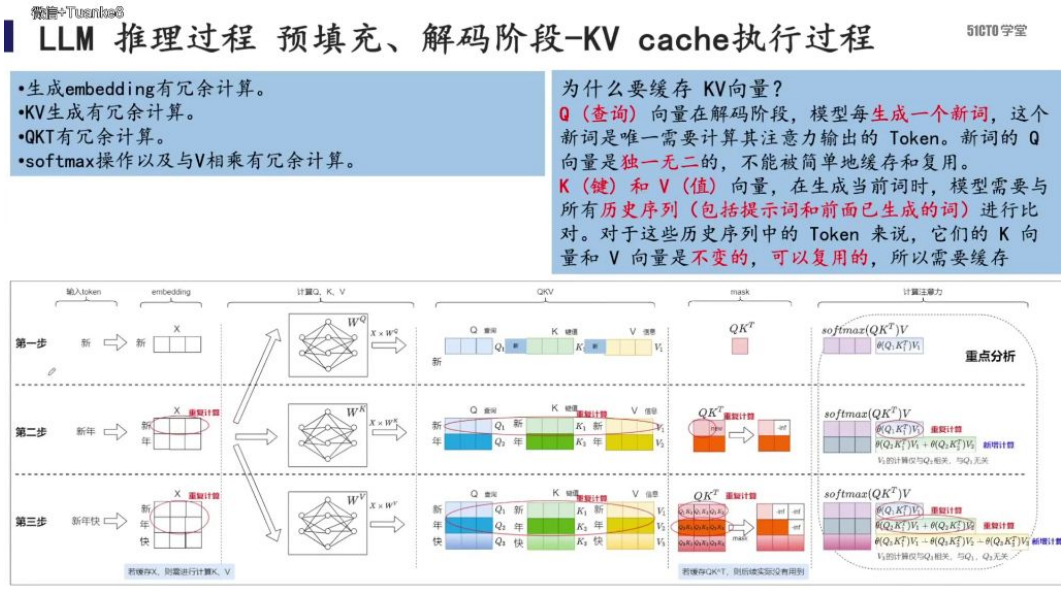
LLM 推理过程 预填充、解码阶段-KV cache执行过程

51CTO学堂



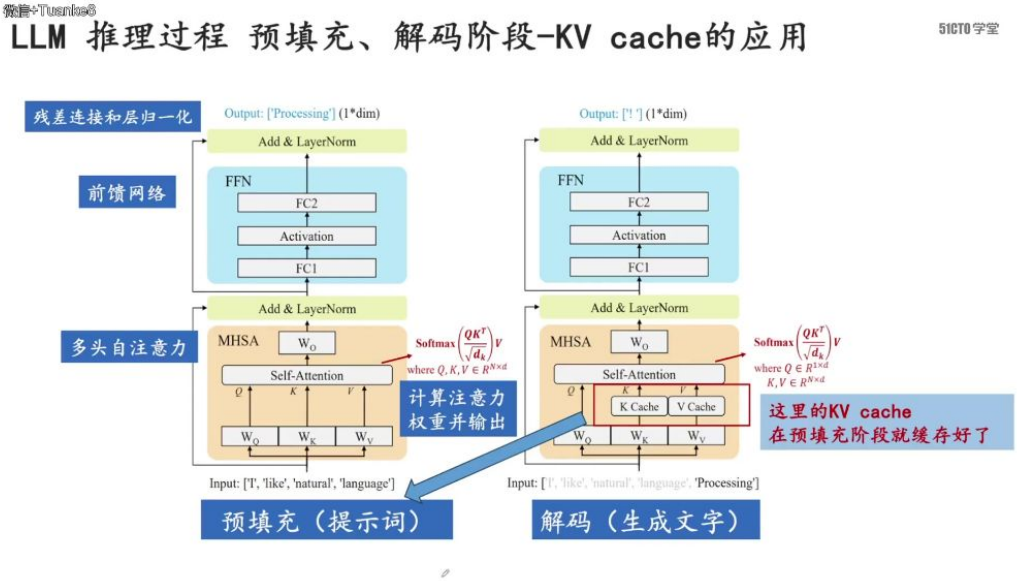
-
- **输入扩展**：拼接"新"、"年"、"快"作为输入
- **向量复用**：
  - 复用"新"的K_1、V_1
  - 复用"年"的K_2、V_2
  - "快"生成新的Q_3、K_3、V_3
- **复合计算**：公式扩展为$\theta(Q_3K_1^T)V_1 + \theta(Q_3K_2^T)V_2 + \theta(Q_3K_3^T)V_3$
- **输出生成**：最终输出"乐"字完成"新年快乐"的生成

4. 总结重复计算 08:20



- **冗余计算类型**：
  - Embedding重复计算
  - KV向量重复生成
  - QK^T重复计算
  - softmax及与V相乘的重复计算
- **缓存必要性**：历史token的K、V向量不变且被反复使用，每次只需计算新token的Q向量

三、kv cache的应用 12:07



- **缓存机制**：
  - 预填充阶段：提前计算并缓存提示词（如"I like natural language"）的K、V向量
  - 解码阶段：直接复用缓存的K、V，仅计算新生成token的Q向量
- **优化效果**：避免重复计算历史token的键值向量，显著提升推理效率
- **架构支持**：Transformer解码器通过残差连接和层归一化(Add & LayerNorm)保持计算稳定性

四、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Transformer解码阶段的自回归生成机制	通过逐步生成token（如“新年快乐”），每个新token依赖前序token的q/k/v向量计算，并重复拼接历史输入	**自回归特性**导致历史token的k/v向量重复计算	★★★
注意力机制的三向量运算	q（查询）、k（键）、v（值）通过公式 $\text{softmax}(q \cdot k^T/\sqrt{dim}) \cdot v$ 生成注意力输出	**q/k/v来源**：当前输入通过嵌入层与权重矩阵生成	★★★★
KV缓存（KV Cache）原理	历史token的k/v向量可复用，避免重复计算；**仅缓存k/v**因q需实时生成新token的查询向量	**缓存范围**：预填充阶段生成的k/v直接用于解码阶段	★★★★
分步推理示例（生成“新年快乐”）	1. 输入“新”→输出“年” 2. 输入“新年”→输出“快” 3. 输入“新年快”→输出“乐”	**冗余计算**：红框标注的k/v运算在每步重复出现	★★★
掩码（Mask）机制	防止模型“偷看”后续token，通过掩码遮盖未生成部分	仅影响softmax前的权重分配	★★